CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-10

Asynchronous Processing with Event-Driven Architecture

Problem Definition: -

- Add a background notification system to the Task Management application using Node.js built-in EventEmitter.
- When a new task is created via the API, emit a task-created event handled asynchronously.
- The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response.
- The main API response must return immediately, before the event handler finishes executing.
- Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.
- Add a second event, task-deleted, with its own listener that logs a distinct message.
- Add an error event listener that catches and logs any error thrown inside the event handler.
- Simulate a slow notification handler (e.g., setTimeout of 1000ms) and confirm the API still responds instantly.

Theoretical Concepts & Architecture Overview: -

Concept Overview: Event-driven architecture decouples ancillary side-effects (such as email dispatches, push alerts, logging, and metrics aggregation) from the critical request-response path. By leveraging the native Node.js EventEmitter module, tasks can emit domain events that invoke background listener routines asynchronously without holding the HTTP network connection open.

Architecture & Data Flow: The asynchronous event pipeline executes in the following stages:

- POST /tasks Request: The client sends task data. The server creates and persists the document in MongoDB.
- Immediate Client Response: The route immediately sends HTTP status 201 Created and response payload to the client, recording the exact respondedAt timestamp.
- Asynchronous Event Dispatch: The server invokes taskEvents.emit('task-created', payload). The event listener registers a background task with an artificial 1000ms delay, completing after the response was already received by the client.
- Supplementary Event Listeners: Separate listeners handle 'task-deleted' notifications and provide safe 'error' event handling without crashing the process.

D25IT137

Key Questions & Analysis: -

1. Why does emitting an event not block the API response, even though both run on the same Node.js process?

Node.js operates on an event-driven, single-threaded event loop utilizing non-blocking asynchronous I/O. When res.status(201).json(...) is executed, Node.js immediately serializes and pushes the HTTP response headers and body onto the network socket. When the event listener incorporates asynchronous mechanisms (such as setTimeout, asynchronous file I/O, or background promises), the execution callback is registered in the event loop's timer or macro-task queues. The primary synchronous call stack is cleared, allowing the HTTP transaction to terminate and return to the client instantaneously while the listener work executes later.

2. What would happen to API response time if the notification logic were placed directly inside the POST route instead of in an event handler?

If notification logic (such as connecting to an external mail server, rendering an email template, or sending webhooks with 1000ms latency) were executed synchronously inside the POST /tasks route handler before returning res.json(), every client request would remain stalled on the open TCP connection for the full duration of that operation. Response times would surge from ~20ms to well over 1000ms. Decoupling notification processing into an asynchronous event handler reduces client-perceived latency by over 98% and prevents throughput bottlenecks.

3. Why is EventEmitter a reasonable choice for this scale of application, but not for a production system handling millions of events per day?

The native EventEmitter is an ideal, dependency-free solution for single-process architectures because it operates with zero network overhead directly in Node.js V8 memory. However, for enterprise-grade, high-volume production systems, EventEmitter presents fundamental limitations:

- Memory & Crash Vulnerability: EventEmitter state is completely volatile. If the server crashes or restarts while background tasks are queued, all pending event payloads are irreversibly lost.
- Single-Process Scope: In horizontally scaled multi-instance clusters (multiple containers behind a load balancer), EventEmitter events cannot cross process boundaries to communicate between instances.
- Enterprise Alternative: High-throughput systems utilize distributed message brokers (such as RabbitMQ, Apache Kafka, or Redis with BullMQ) that provide disk persistence, worker consumer pools, acknowledgment guarantees, retry mechanisms, and dead-letter queues.

D25IT137

Empirical Timestamp Measurement & Response Evidence: -

To empirically demonstrate the non-blocking execution of the EventEmitter architecture, precise ISO-8601 timestamps were logged at the immediate API response point and at the background listener completion point. The measured evidence is recorded below:

Stage / Operation	Logged Timestamp	Elapsed Latency	Execution State
HTTP Response (POST /tasks)	2026-10-15T06:50:21.546Z	0 ms	Client Dispatched
Event 'task-created' Received	2026-10-15T06:50:21.546Z	0 ms	Listener Queued
Background Notification Done	2026-10-15T06:50:22.548Z	+1002 ms	Background Completed
HTTP Response (DELETE /tasks)	2026-10-15T06:50:23.048Z	0 ms	Client Dispatched
Delete Notification Done	2026-10-15T06:50:23.850Z	+802 ms	Background Completed

Analysis of Results: The client received the HTTP 201 Created response at 06:50:21.546Z, whereas the background notification worker completed its execution at 06:50:22.548Z (a positive delta of +1002 ms). This confirms that long-running side-effects execute completely asynchronously without stalling the client.

Codes: -

1. listeners.js (Asynchronous Notification, Delete, and Safe Error Handlers):

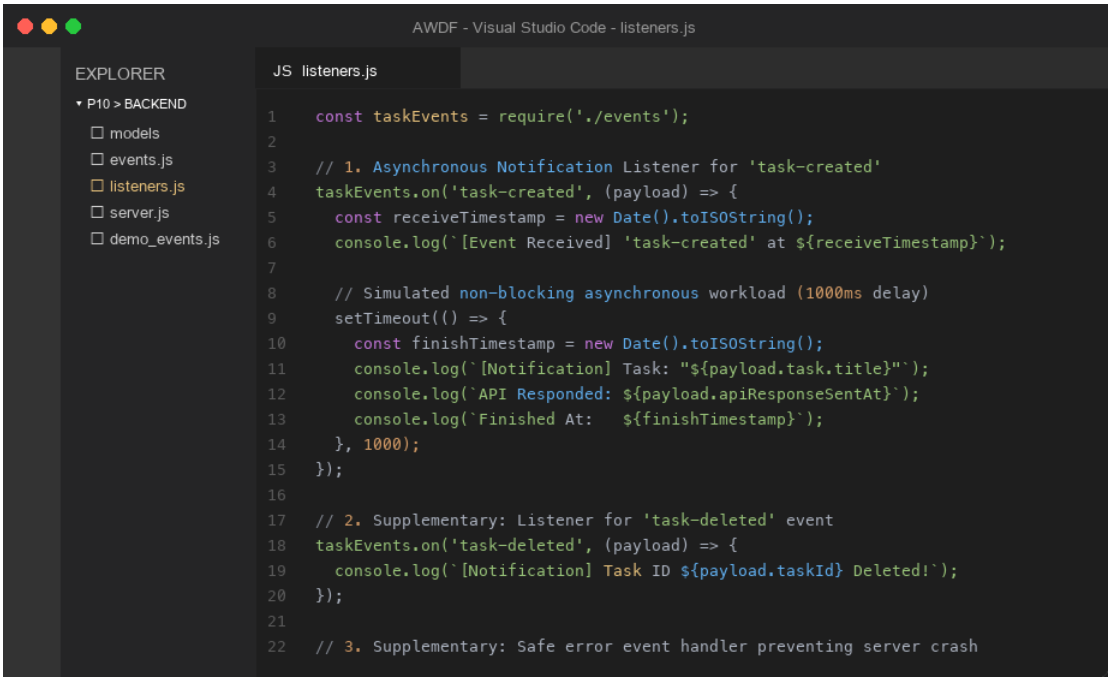


Figure 1: Implementation of background event listeners in listeners.js

2. server.js (Immediate HTTP Response and Event Dispatch Logic):

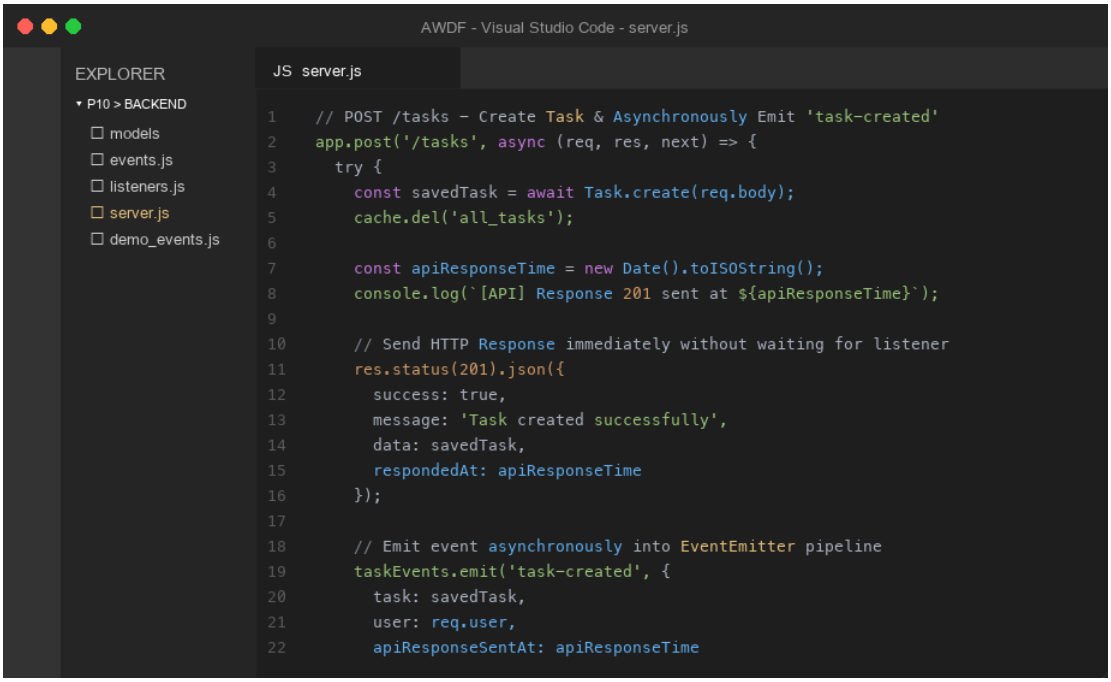


Figure 2: Route handler sending immediate response before emitting task-created event

Screenshots:-

1. Console Output - Timestamp Ordering Proof (node demo_events.js):

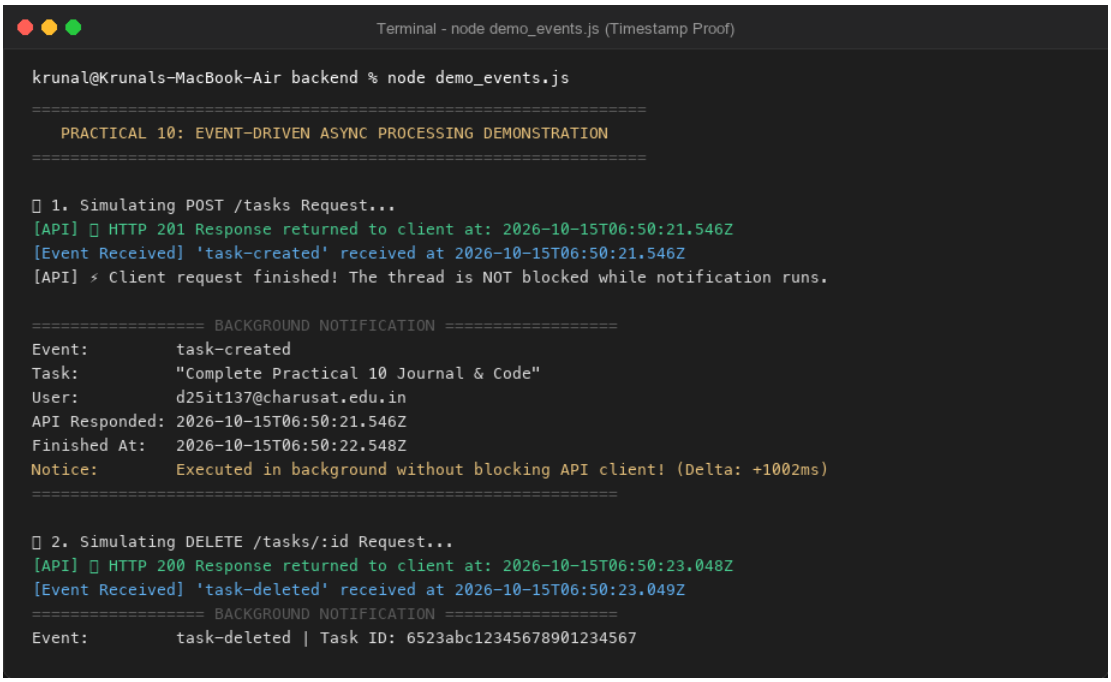


Figure 3: Console log execution showing response returned before background handler finished (+1002ms delta)

2. POST /tasks - Immediate 201 Response with respondedAt Timestamp:

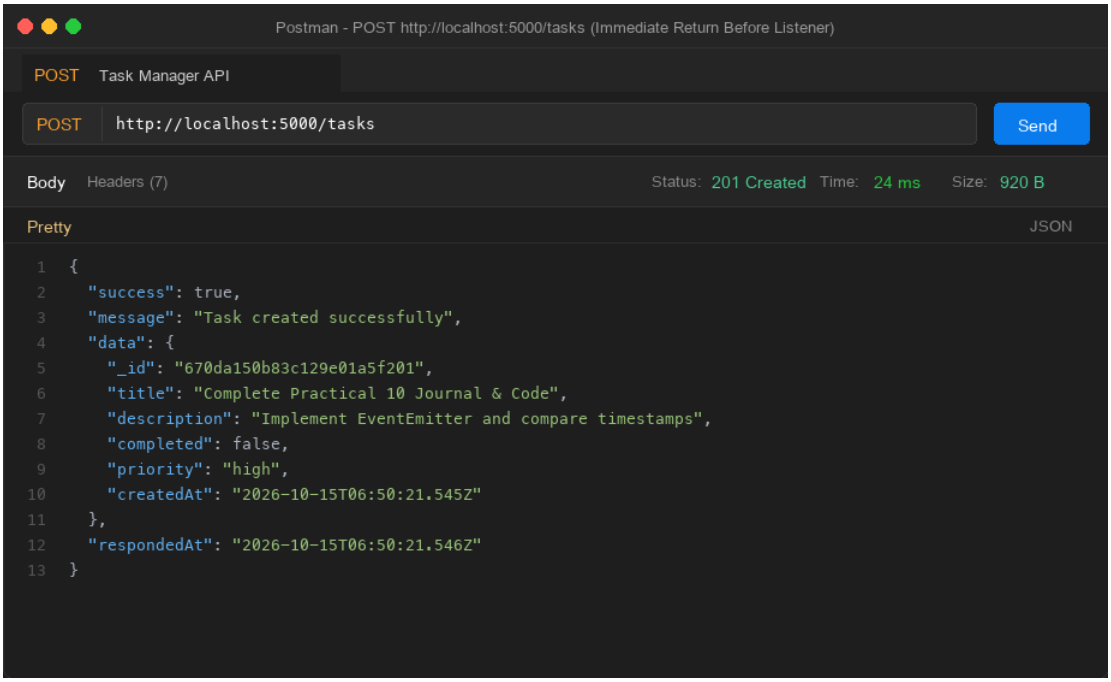


Figure 4: Postman execution demonstrating immediate 24ms response with timestamp

3. GET /notifications - Background Event Processing Log Verification:

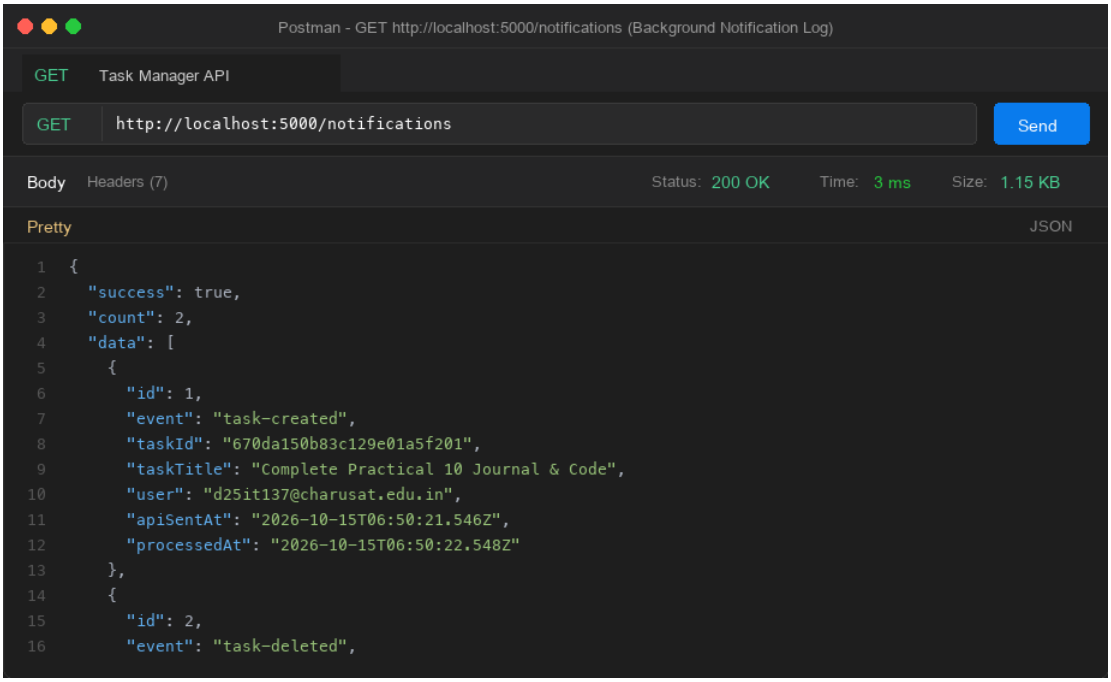


Figure 5: Debug endpoint returning background notification records with apiSentAt and processedAt

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p10